

4장_딥러닝

인공 신경망의 한계와 딥러닝 출현

퍼셉트론(perceptron)

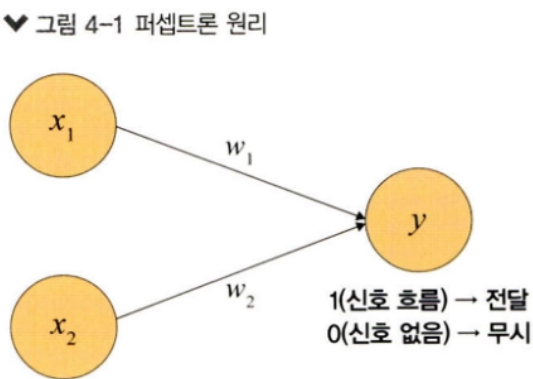
- 인간의 논리 회로를 모방하여 이를 논리 회로 모델로 구성한 것

뉴런의 이해

- 가지돌기에 자극을 받게 되면, 신경세포체를 통해 축삭돌기에 화학물질이 전달되어, 전이가 발생하고
- 전이의 정도가 특정 임계값을 넘으면, 시냅스를 통해, 신호를 다음 뉴런에 전달하는 구조

인공신경망(artificial neural network, ANN)

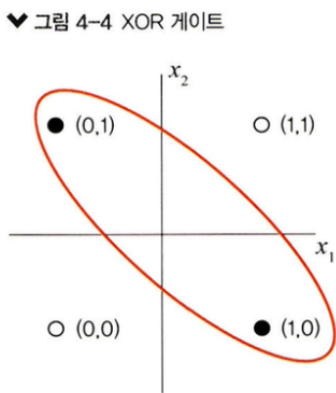
- 뉴런의 작용을 0과 1로 이루어진 이진법 논리 회로 모델로 구성 → 이런 모델을 그물망처럼 연결하면, 인간의 뇌처럼 기계를 동작시킬 수 있을 것 (= 퍼셉트론)



- $x_0, x_1, x_2, \dots, x_n$ 의 각각의 입력값을 받아서,
- $x_0 * w_0 + b, x_1 * w_1 + b, \dots, x_n * w_n + b$ 각 값을 가중치와 곱해서 (bias도 더함)
- $\sum_{i=1}^n (x_i * w_i + b)$ 이 특정 임계값을 넘으면 1, 그렇지 않으면 0과 같은 알고리즘으로 구성됨
- 예: if $\sum_{i=1}^n (x_i * w_i + b) > 0$ then 1, else 0
- 실제 결과와 오차가 생기면, 가중치를 업데이트함
 - 실제 결과와의 오차를 기반으로 가중치 업데이트를 할 수 있는 구조로 인해, 최초로 학습이 가능하게 됨

퍼셉트론의 이슈

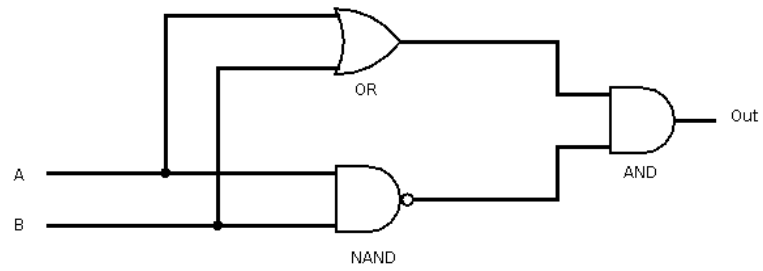
- 조건을 만족하는 선이 여러개 나와서, 최적의 선 구하기 어려움
 - [해결책] Adaline(아달라인) : 경사하강법을 도입하여 가중치를 업데이트시킴
- 하나의 레이어를 가지고, 0과 1을 출력하는 퍼셉트론의 직선형태의 구분선으로는 XOR문제를 풀 수 없음



▼ 표 4-3 XOR 게이트

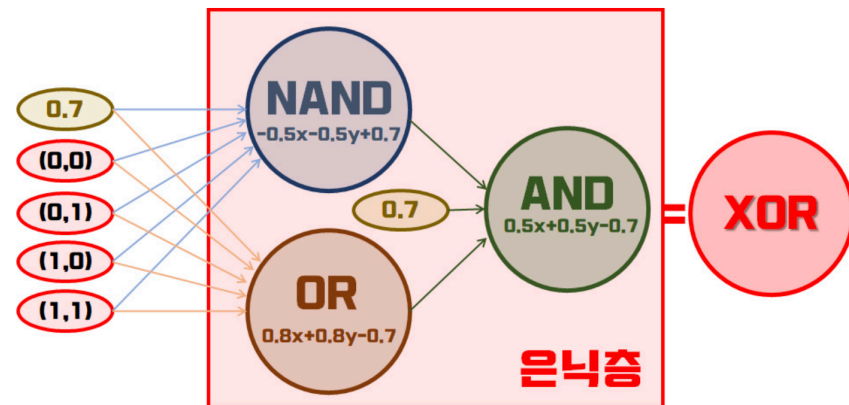
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

- [해결책] NAND, OR, AND 게이트를 조합하면 XOR 게이트를 만들 수 있음



[출처] <http://sullystationtechnologies.com/npnxorgate.html>

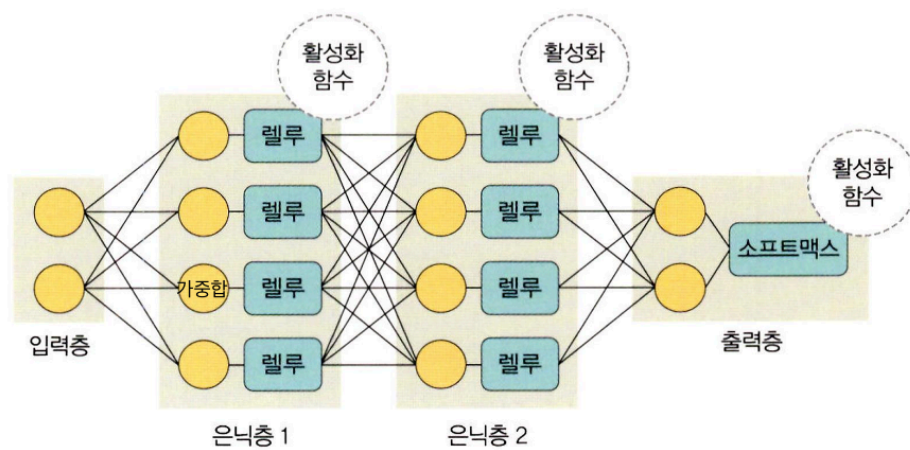
⇒ 즉, 여러 퍼셉트론 레이어(= 다중 퍼셉트론)를 두어, 은닉층을 만들어서 해결



XOR을 구현한 다중 퍼셉트론의 예시

⇒ 이때, 은닉층이 2개 이상인 신경망을 DNN(Deep Neural Network)라고 함

딥러닝 구조

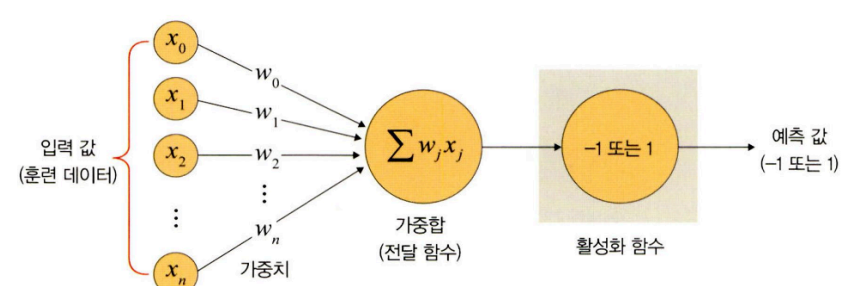


구분	구성요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결과값이 포함된 층

가중치

- 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소
 - w_1 값이 0 혹은 0과 가까운 0.001이라면, x_1 이 아무리 큰 값이라도, $x_1 * w_1$ 값은 0이거나 0에 가까운 값이 됨

▼ 그림 4-6 가중치

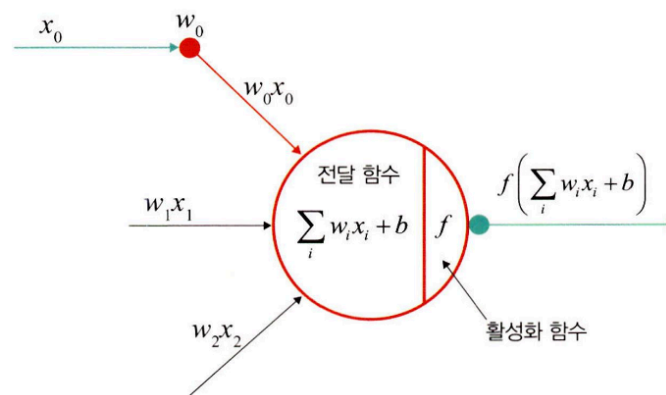


→ 이와 같이 연산 결과를 조정하는 역할을 하는 것이 가중치

가중합 또는 전달 함수

- 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달되는데, 이 값들을 모두 더한 합계
 - $\sum_{i=1}^n (x_i * w_i + b)$
- 노드의 가중합이 계산되면, 이 가중합을 활성화 함수로 보내기 때문에 전달 함수(transfer function)이라고도 함

▼ 그림 4-7 전달 함수



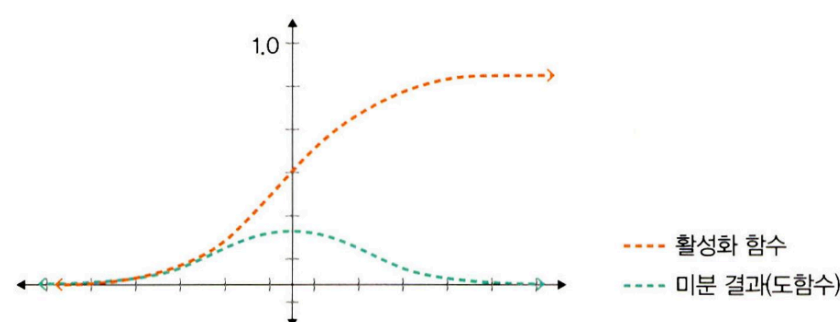
활성화 함수

- 가중합(전달 함수)에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수
- 세상 다양한 문제는 non-linear 문제가 더 많음 → 다층 퍼셉트론에서 activation function으로 non-linear function을 사용하여 non-linear 문제를 보다 잘 모사할 수 있도록 구성

1. Sigmoid 함수

- 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형해 줌
 - 수식 : $\text{sigma}(x) = \frac{1}{1+e^{-x}}$
- sigmoid 함수는 머신러닝의 주요 모델 중 하나인 logistic regression 모델에서 많이 사용됨 → logistic regression은 이름에 regression이 들어 있지만 분류(Classification) 모델임
- **Logistic Regression**
 - Linear Classification(직선 모양)과 달리, **"S"**자 모양으로 곡선으로 계산하는 **Sigmoid function**을 사용해서 **분류**하는 모델
 - 정확히 0 또는 1을 예측하는 대신 확률(0과 1 사이의 값)을 생성해서 0과 1을 분류하는 예측 모델
- **Gradient Vanishing**가 발생하여, 현재 딥러닝 모델에서는 잘 사용되지 않음

▼ 그림 4-8 시그모이드 활성화 함수와 미분 결과

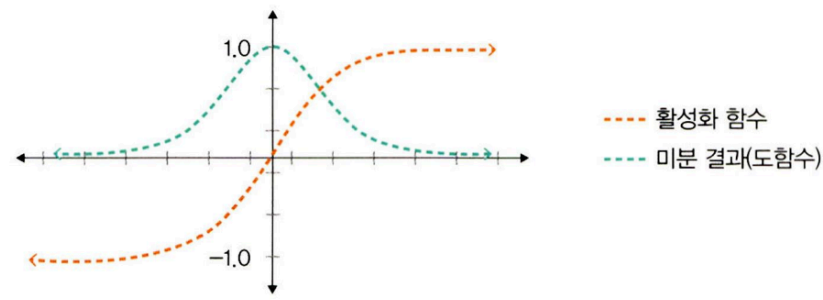


2. hyperbolic tangent 함수

- 선형 함수의 결과를 -1 ~ 1 사이에서 비선형 형태로 변형해 줌

- Sigmoid에서 결과값의 평균이 0이 아닌 양수로 편향된 분제를 해결하는데 사용했지만, Gradient Vanishing 문제는 여전히 발생

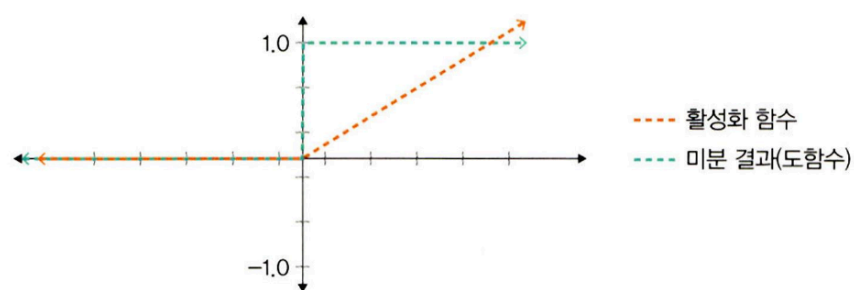
▼ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과



3. ReLU 함수

- **Gradient Vanishing** 이슈를 해결하기 위해 고안된 **activation function**
- 수식: $f(x) = \max(0, x)$
 - 입력값 x 가 양수일 경우: $f(x) = x$
 - 입력값 x 가 0 이하일 경우: $f(x) = 0$
 - 출력 범위: $[0, \infty)$
- ReLU 함수의 미분은 $x \geq 0$ 일 때는 1, $x < 0$ 일 때는 0 이므로
 - 경사 하강법(gradient descent)에 영향을 주지 않아 학습 속도가 빠르고, Gradient vanishing 이슈를 어느 정도 보완할 수 있음
 - $x \geq 0$ 이면 미분이 1이므로 ,학습 속도가 sigmoid 보다 빠른 편임
 - sigmoid는 x 값이 커지면, y값이 1에 가까워지므로 기울기가 낮아져서 → 일정 값 이상이 되면 학습속도가 현저히 낮아짐
 - 일반적으로 은닉층에서 사용되며, hyperbolic tangent 함수 대비 학습 속도가 6배 빠름
- **ReLU 함수는 $x < 0$ 일 때는 미분값이 0이 됨 → 학습 능력이 감소함**
 - 가중치값으로 계산된 음수일 경우, 가중치 업데이트가 멈출 수 있음 (미분값이 0이기 때문)

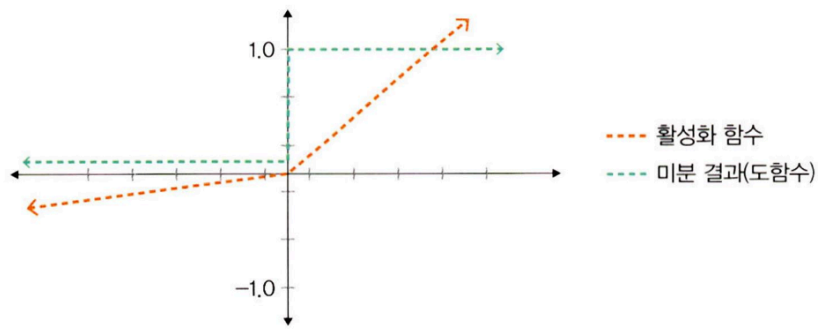
▼ 그림 4-10 렐루 활성화 함수와 미분 결과



4. Leaky ReLU 함수

- **ReLU 함수의 문제를 해결하기 위해 나온 방안으로, 입력 값이 음수이면 0이 아닌 0.001처럼 매우 작은 수를 반환**
 - 입력 값이 수렴하는 구간이 제거되어 ReLU 함수를 사용할 때 생기는 문제를 해결
- 수식: $f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$

▼ 그림 4-11 리키 렐루 활성화 함수와 미분 결과



5. Softmax 함수

- 입력값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
 - 보통 딥러닝에서 출력 노드의 활성화 함수로 많이 사용됨
- 수식 : $y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$
 - $\exp(x)$: 지수 함수
 - n : 출력층의 뉴런 개수, y_k : 그 중 k번째 출력
 - 소프트맥스 함수의 분자는 입력 신호 a_k 의 지수함수, 분모는 모든 입력 신호의 지수 함수 합으로 구성됨

손실 함수

- **Gradient descent(경사 하강법)**은 학습률(learning rate)과 손실 함수(loss function)의 순간 기울기를 이용하여 가중치를 업데이트 하는 방법
 - 즉, 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법
 - **손실 함수(오차를 구하는 방법, 표본으로부터 추정된 예측값과 실제값의 차이)**는 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표
 - 값이 클수록 많이 틀렸고, 0에 가까울 수록 완벽하게 추정할 수 있다는 의미

1. 평균 제곱 오차 (Mean Squared Error, MSE)

- 수식 : $MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$
 - N : 샘플 개수
 - y_i : 실제값 (정답 레이블)
 - $\hat{y}_i$: 모델의 예측값

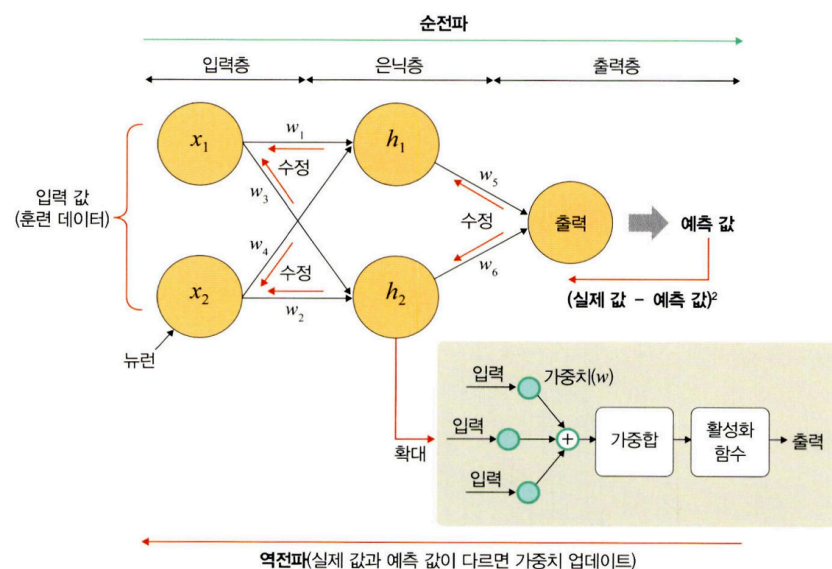
2. 크로스 엔트로피 오차 (Cross Entropy)

- 크로스 엔트로피 오차(CEE)는 **분류(classification) 문제에서 원-핫 인코딩(one hot encoding)**했을 때만 사용할 수 있는 오차 계산법
- 일반적으로 분류 문제에서는 데이터의 출력을 0과 1로 구분하기 위해 Sigmoid 함수를 사용하는데, 이 함수에 포함된 자연 상수 e 때문에 MSE를 적용하면 매끄럽지 못한 그래프(울퉁불퉁한 그래프)가 출력되어 크로스 엔트로피 손실 함수를 적용함
- 크로스 엔트로피 오차 손실 함수를 적용할 경우, Gradient descent 과정에서 학습이 local minimum에서 멈출 수 있기 때문에, 이를 방지하고자 자연 상수 e에 반대되는 자연 로그를 모델 출력 값에 취함

- 수식 : $CE = - \sum_{i=1}^N y_i \log \hat{y}_i$
 - C : 클래스 개수
 - y_i : 원-핫 인코딩된 실제값 (해당 클래스이면 1, 아니면 0)
 - $\hat{y}_i$: 모델이 예측한 클래스 의 확률값

딥러닝 학습

▼ 그림 4-12 순전파와 역전파



1. 순전파(feedforward)

- 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환(가중합 및 활성화 함수)을 적용하여 다음 층(은닉층)의 뉴런으로 전송하는 방식

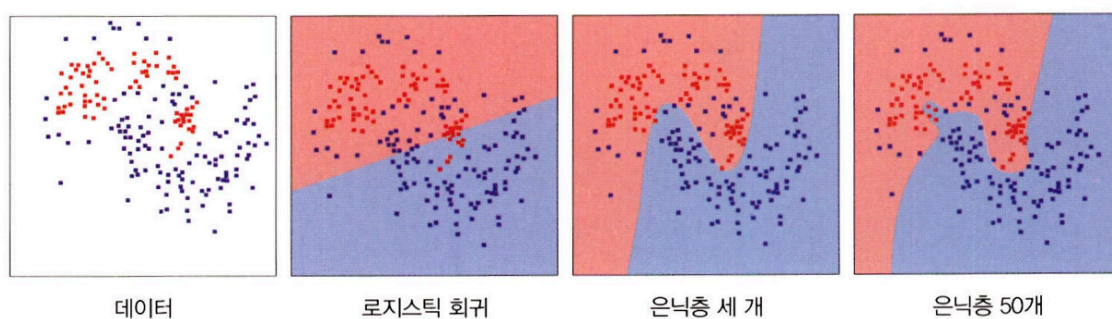
2. 역전파(backpropagation)

- 손실 함수로 네트워크의 예측 값과 실제 값의 차이(손실, 오차)를 추정 → 이때 손실 함수 비용이 0에 가깝도록 하기 위해 모델이 훈련을 반복하면서 가중치를 조정
 - 손실(오차)가 계산되면 그 정보는 역으로 전파(출력층→은닉층→입력층)되기 때문에 역전파
 - 출력층에서 시작된 손실 비용은 은닉층의 모든 뉴런으로 전파
 - but, 은닉층의 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도에 따라(가중치에 따라) 값이 달라짐
- ⇒ 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 뺌 → 이 과정을 출력층→은닉층→입력층 순서로 모든 뉴런에 대해 진행하여 계산된 각 뉴런의 결과를 또 다시 순전파의 가중치 값으로 사용

딥러닝의 문제점과 해결 방안

- 활성화 함수가 적용된 은닉층의 개수가 많을수록 데이터 분류가 잘 되지만, 은닉층이 많을 수록 세 가지 문제점이 생김

▼ 그림 4-13 은닉층이 분류에 미치는 영향

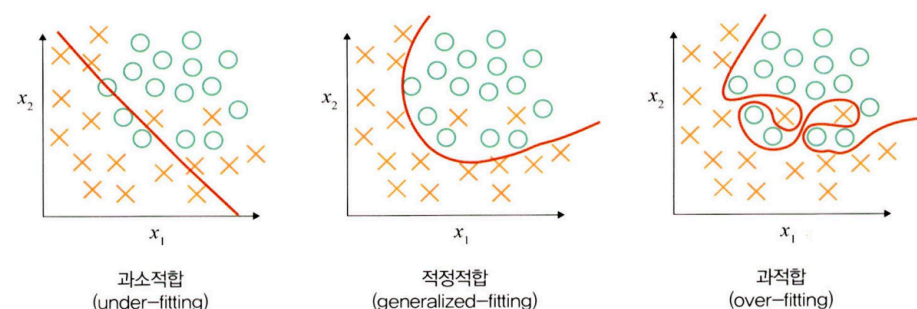


문제점

1. 과적합 문제(over-fitting) 발생

- 훈련 데이터에 대해 과하게 학습하여 실제 데이터에 대한 오차가 증가하는 현상

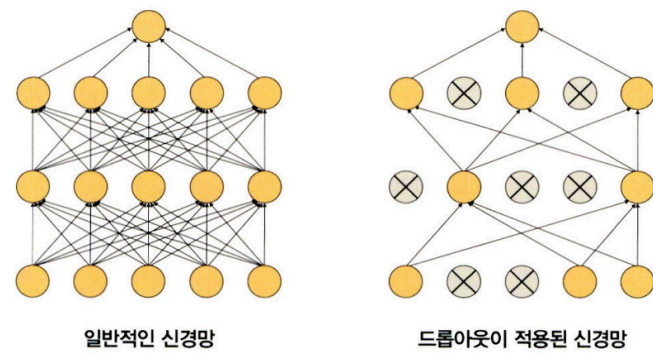
▼ 그림 4-14 과적합



- [해결방안] dropout

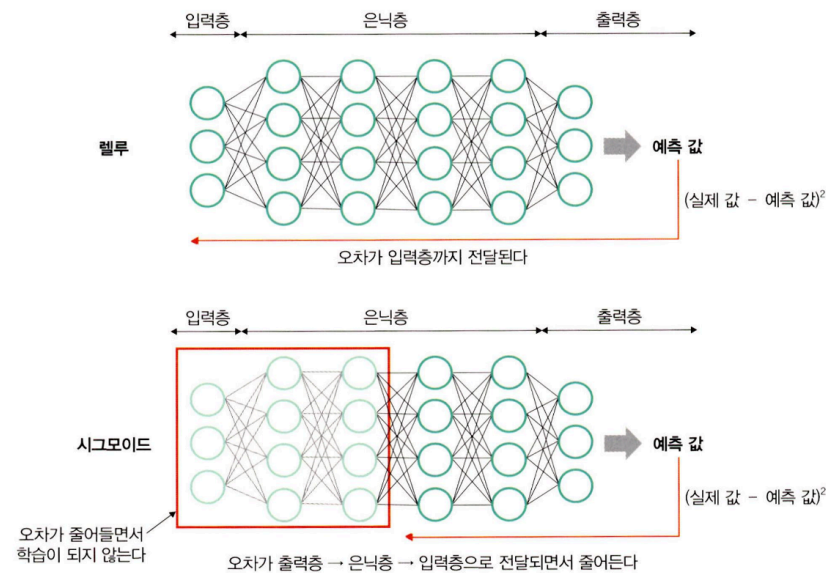
- 신경망 모델이 과적합되는 것을 피하기 위한 방법 → 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴

▼ 그림 4-15 일반적인 신경망과 드롭아웃이 적용된 신경망



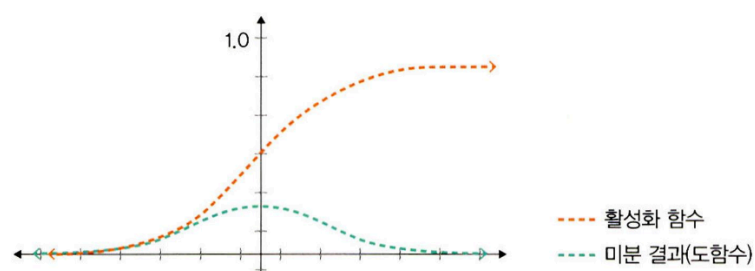
2. 기울기 소멸(Gradient Vanishing) 문제 발생

▼ 그림 4-16 기울기 소멸 문제



- **다층 퍼셉트론에 오차역전파 적용 시, 입력층으로 갈수록 기울기가 점차적으로 작아지는 현상** → 따라서, 입력층으로 갈수록 빠주는 값 자체가 작아져서(= 입력층으로 갈수록 그 가중치가 제대로 업데이트가 되지 않는 현상)
 - (↔ Gradient Exploding : 입력층에 갈수록 기울기가 비정상적으로 커져서 발산하는 현상)
- [이유] (ex) sigmoid 함수 : 결과값을 0과 1 사이의 값으로 조정하여 반환
 - 출력층의 마지막에 sigmoid 함수를 넣을 수도 있지만, 각각의 레이어마다도 넣을 수 있음
 - sigmoid가 중간에 오차역전파를 할때도, 역시 연산에 포함되어있기 때문에 sigmoid 함수 자체도 미분이 됨
 - sigmoid 미분은 무조건 1보다 작음 → $x < 1$ 이면, x^{10} , x^{100} 과 같이 x 를 곱할 수록, 최종값은 0에 수렴하게 됨

▼ 그림 4-8 시그모이드 활성화 함수와 미분 결과

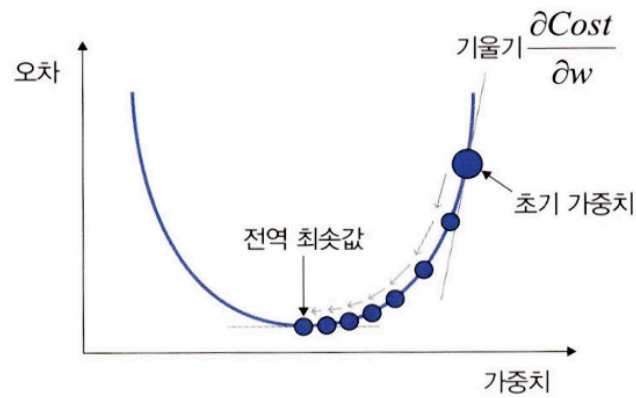


- activation function 미분 값이 1보다 작을 경우, 레이어가 입력에 가까울 수록, 해당 값이 0에 가까워질 수 있음 → 결국 각 가중치 업데이트는 $w^{100} := w^{100} - \alpha \frac{\partial \text{Loss}}{\partial w^{100}}$ 이기 때문에, 가중치가 업데이트되기 어려움
- sigmoid 이외에도, **적용한 activation function의 미분 값이 1보다 작은 경우, Gradient Vanishing 현상이 발생 가능**

3. 성능이 나빠지는 문제 발생

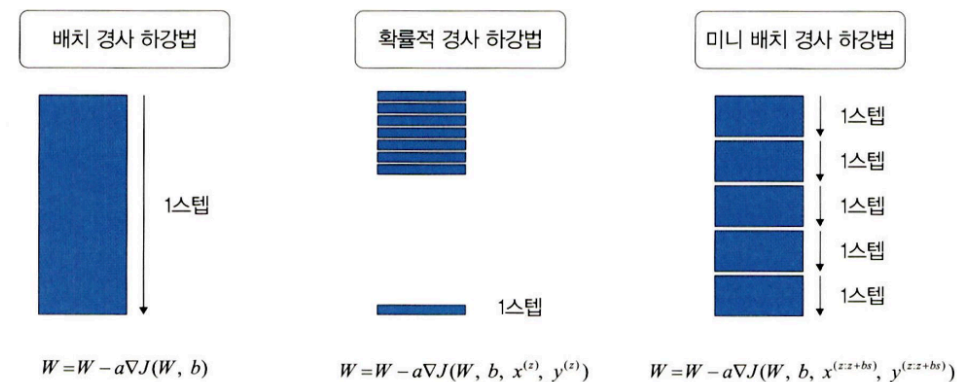
- 경사하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복 → 이때 성능이 나빠지는 문제 발생

▼ 그림 4-17 경사 하강법



- [해결방안] : 확률적 경사 하강법, 미니 배치 경사 하강법

▼ 그림 4-18 경사 하강법의 유형



1. 배치 경사 하강법 : 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트 → 전체 훈련 데이터셋에 대해 가중치를 편미분
2. 확률적 경사 하강법 : 임의로 선택한 데이터에 대해 기울기를 계산하는 방법
 - 적은 데이터를 사용하기 때문에 빠른 계산 가능
 - 파라미터의 변경 폭이 불안정하고, 때로 배치 경사 하강법보다 정확도가 낮을 수 있지만 속도가 빠른 장점
3. 미니 배치 경사 하강법
 - 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것을 평균 기울기를 이용하여 모델을 업데이트해서 학습하는 방법
 - 전체 데이터를 계산하는 것 보다 빠르고, 확률적 경사 하강법보다 안정적이고 속도도 빠름

옵티마이저(Optimizer)

- 학습 : 손실함수(loss function) 최소값을 찾아가는 과정 = 최적화(Optimization)
- 최적화(Optimization)을 수행하는 알고리즘 = 옵티마이저(Optimizer)

1. BGD (Batch Gradient Descent, 배치 경사 하강법)

- **전체 데이터셋에 대해 한번에 학습 (1 epoch = 1 iteration)**
- 이 때의 Batch는 전체 데이터셋을 의미 → 전체 데이터셋을 기반으로 학습시키기 때문에 모델 파라미터 업데이트에 시간이 상대적으로 오래 걸림(한번 iteration을 도는 데에도 시간이 오래 걸림)
- 전체 데이터셋으로 학습시키기 때문에, 항상 입력이 일정하기 때문에, 수렴되는 정도가 일정함(안정적)
→ 수렴되는 정도가 안정되기 때문에, local minima 상태가 되면 빠져나오기 어려움

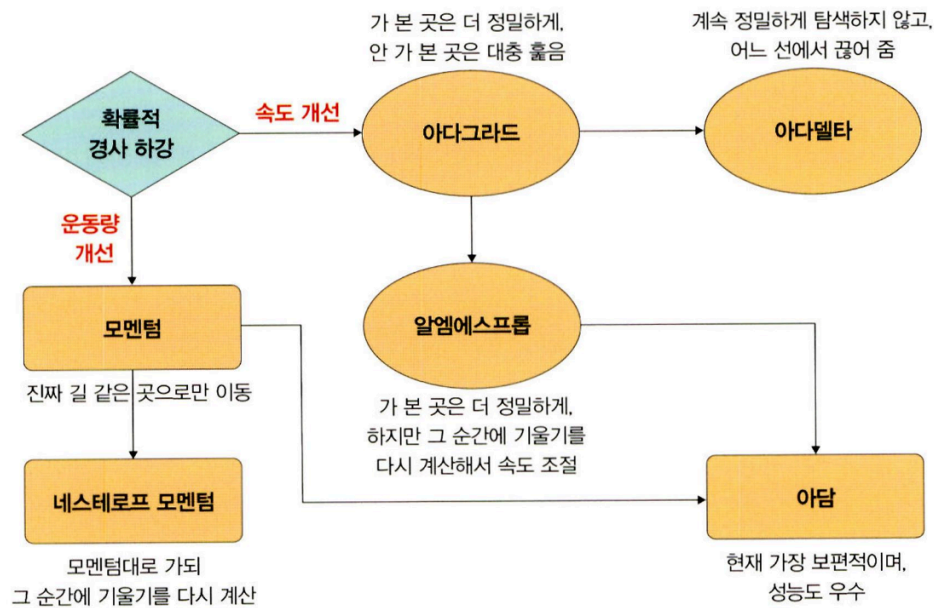
2. SGD (Stochastic Gradient Descent) : 확률적 경사 하강법

- **전체 학습 데이터 중 특정 크기만큼 임의로 선택해서 학습**
 - **미니배치(Mini-Batch)**

전체 데이터셋이 1000개가 있다고 할 때, 이를 200개의 Batch-Size로 분리하면, 각 200개의 서브 데이터셋을 Mini-batch라고 함
→ 1 epoch를 위해서는 200개씩 5번의 iteration이 필요함

- BGD의 단점을 보완하기 위해 고안 → **전체 데이터 중 랜덤하게 선택된 k개의 샘플 데이터를 이용하여 학습시키는 경사 하강법 (Mini-Batch-Gradient Descent)**
 - 원래는 전체 데이터 중 1개의 샘플 데이터만 사용해서 학습시키는 경사하강법이 SGD임 → 최근에는 Mini-Batch-Gradient Descent를 SGD라고 부르는 경향이 있음
 - 전체 데이터가 500개 이고, batch size가 100이라면, 결국 1 epoch에는 5 iteration을 수행하게 되고, 전체 데이터셋에 대해 1000 epoch를 수행하고자 하면, 결국 5 X 1000 iteration의 학습 및 가중치 업데이트가 일어남

속도를 조정하는 방법



1. AdaGrad

- learning rate 값을 변경하는 알고리즘
 - SGD, Momentum, NAG 모두 모든 파라미터 업데이트에 동일한 learning rate 값 적용 → 학습률이 너무 작으면, 학습시간이 너무 길고, 학습률이 너무 크면, 발산해서 학습이 제대로 이루어지지 않음
- AdaGrad는 이를 learning rate decay 기법으로 개선하고자 함

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

- iteration 중, 많이 움직이는 경우(기울기가 컸던 경우), 해당 iteration 의 learning rate 는 그만큼 작아지므로, 학습률이 각 iteration 마다 다르게 적용됨 (각 iteration 별 맞춤형 learning rate 적용이 가능함)
- but, Adagrad는 t 가 증가함에 따라, G_t 가 점점 커져서 learning rate가 점점 소실되는 문제점이 있음

2. Adadelta

- 아다그라드에서 G 값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장한 방법
- 아다그라드의 수식에서 학습률을 D 함수(가중치의 변화량 크기를 누적한 값)로 변환했기 때문에 학습률에 대한 하이퍼파라미터가 필요 없음

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1) + \epsilon}}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1 - \gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1 - \gamma)(\Delta(w(i)))^2$$

3. RMSProp

- Adagrad 에서 발생한 t 가 증가함에 따라 learning rate가 점점 소실되는 문제를 해결
- G 를 구할 때 합을 이용하지 않고, Exponential Weighted Moving Average(지수 가중 평균)를 이용

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

- 시간이 흐름에 따라, 오래된 데이터의 영향이 지수적으로 감소함

운동량을 조정하는 방법

1. Momentum

- SGD에 관성이 추가된 Optimizer → 관성 적용을 위해, velocity term(v)이 추가됨

$$w(i+1) = w(i) - v(i)$$

$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i))$$

- 매 단계마다 gradient가 동일한 부호를 가지면, v의 절대값은 계속 증가하게됨 → 즉, 같은 방향으로 이동하면 할수록 가속도가 붙게됨
- 따라서, local minima에서 gradient가 0에 수렴해도 이전 단계에서 축적된 v값에 의해서 local minima 등을 탈출할 수도 있음

2. NAG

- momentum 방식은 이동을 중지해야하는 지점에 도달해도 momentum에 의해 해당 지점을 지나칠 수 있다는 문제를 가짐 → 이런 문제점을 해결한 것이 NAG
- momentum 계산 시 momentum에 의해 발생하는 변화를 미리 보고 momentum을 결정

$$w(i+1) = w(i) - v(i)$$

$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))$$

- 현재 W_t 의 위치에서 W_{t+1} 의 gradient를 계산하고자 함 → 현재가 아닌 다음 단계의 기울기를 가지고 가중치를 업데이트 하려고 한 것!

속도와 운동량에 대한 혼용 방법

1. Adam

- 가장 많이 사용하는 optimizer → Momentum과 RMSProp를 결합한 느낌

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\epsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1-\gamma_2)(\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

- momentum의 장점인 이전 gradient의 경향을 적용
- RMSProp의 장점인 learning rate가 점점 소실되는 문제 해결 → 이전 learning rate의 경향을 적용

딥러닝을 사용할 때 이점

-

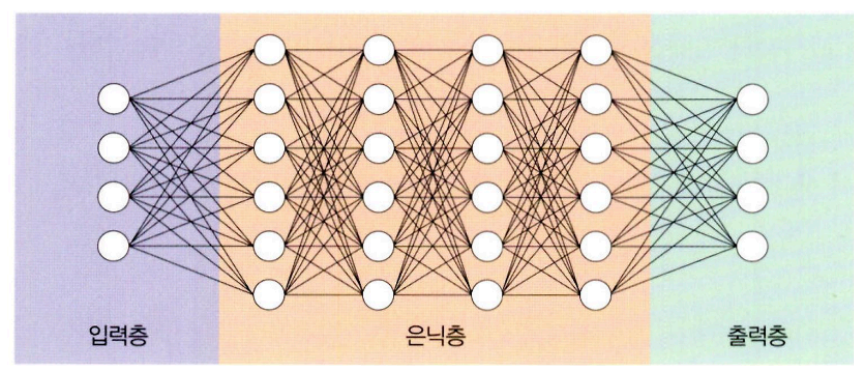
특성 추출 : 데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조
→ 특성 추출 과정을 알고리즘에 통합시킴

- 빅데이터의 효율적 활용 : 특성 추출을 알고리즘에 통합시키는 것이 가능한 이유는 빅데이터 덕분에
→ 딥러닝 학습을 이용한 특성 추출은 사례가 많을 수록 성능이 향상됨

딥러닝 알고리즘

심층 신경망 (DNN)

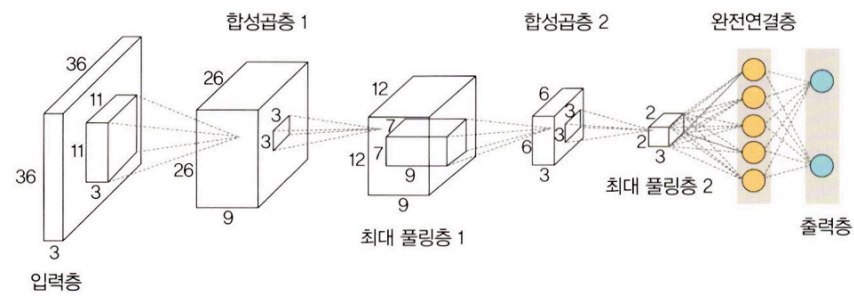
▼ 그림 4-24 심층 신경망



- 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망
 - 다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류가 가능
 - 학습을 위한 연산량이 많고 Gradient Vanishing 문제가 발생할 수 있음
 - [해결방안] dropout, ReLU, 배치 정규화 등을 적용해야함

합성곱 신경망 (CNN)

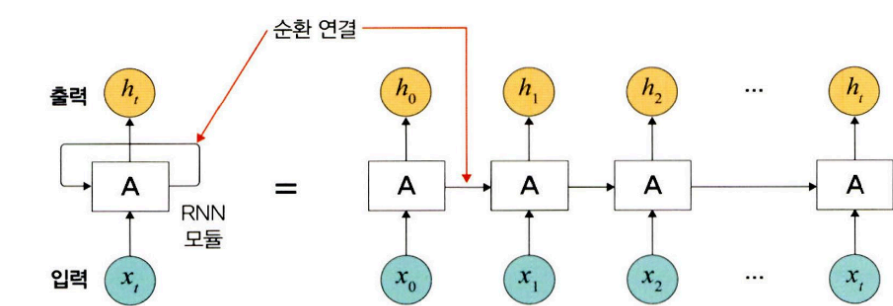
▼ 그림 4-25 합성곱 신경망



- 합성곱층(convolutional layer)과 풀링층(pooling layer)을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘
 - 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용

순환 신경망 (RNN)

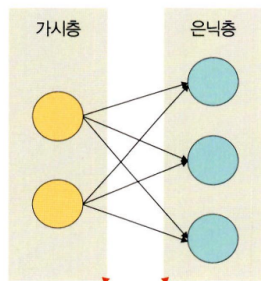
▼ 그림 4-26 순환 신경망



- 시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
 - '순환'은 자기 자신을 참조한다는 것으로 → 현재 결과가 이전 결과와 연관이 있다는 의미

제한된 볼츠만 머신 (RBM)

▼ 그림 4-27 제한된 볼츠만 머신

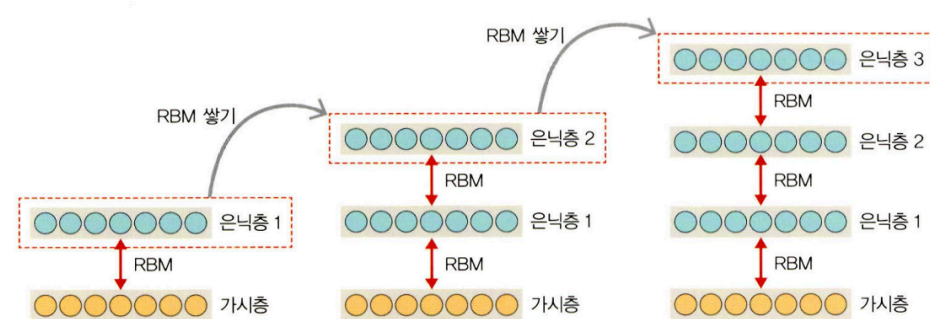


가시층 간, 은닉층 간에 연결은 없다

- 가시층(visible layer)과 은닉층(hidden layer)으로 구성된 모델
 - 가시층은 은닉층과만 연결 (가시층 간, 은닉층 간 연결은 없음)
- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링(collaborative filtering), 특성 값 학습(feature learning), 주제 모델링(topic modeling)에 사용
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
- 심층 신뢰 신경망(DBN)의 요소로 활용

심층 신뢰 신경망 (DBN)

▼ 그림 4-28 심층 신뢰 신경망



- 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망
 - 사전 훈련된 제한된 볼츠만 머신을 층층이 쌓아 올린 구조 → 레이블이 없는 데이터에 대한 비지도 학습이 가능
 - 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 사용하면 유용
- 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 계층적 구조 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성을 추출
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용